

Tipe Data Statis, Operator, dan Assignment

Dalam pemrograman komputer, setiap nilai yang dihasilkan dari sebuah ekspresi memiliki tipe data tertentu. Python memiliki dua jenis tipe data, yaitu tipe data statis dan tipe dinamis. Tipe data ini menunjukkan jenis atau kategori suatu nilai. Seperti halnya di dalam matematika, dikenal beberapa jenis bilangan yaitu misalnya bilangan bulat (... , -3, -2, -1, 0, 1, 2, ...) atau bilangan riil (-3.5, -5.123, 0.234, 3,123, dll).

Di dalam Python, terdapat 3 tipe data statis yaitu sebagaimana terdapat dalam Tabel 4.1. Adapun pembahasan detail tentang tipe data dinamis akan disajikan di bab tersendiri nantinya.

Tabel 4.1 Tipe data utama di Python

Tipe Data	Contoh
Integer (bilangan bulat)	-2, -1, 0, 1, 2
Floating Point (bilangan riil)	-1.212, 0.341, 5.234
String	'a', 'aa', 'python', 'rosihan ari', '(0271) 783243', 'K3517001'
Boolean	True, False

Apabila diperhatikan untuk tipe data string, maka ciri khusus yang menandakan sebuah nilai bertipe data string adalah adanya tanda petik yang mengapitnya. Dengan demikian jika diberikan dua buah nilai 1 dan '1', keduanya memiliki tipe data yang berbeda. Dalam hal ini nilai 1 adalah bertipe data integer, sedangkan '1' bertipe data string.

Sebagai tambahan, tipe data string biasanya dipakai untuk nilai-nilai yang terdiri dari rangkaian satu atau lebih karakter. Misalnya nama orang, alamat, nama benda dll.

Operator

Dalam pemrograman, seringkali terdapat suatu proses yang mengoperasikan beberapa nilai sehingga dihasilkan nilai yang baru. Sebagai contoh misalkan mengoperasikan penjumlahan dari dua bilangan yaitu 4 dan 5. Hasil operasi dari 4+5 adalah 9. Tanda + dalam operasi 4+5 disebut juga operator. Demikian pula tanda -, *, / dan sebagainya itu semuanya adalah termasuk operator. Jadi dalam hal ini fungsi operator adalah mengoperasikan dua atau lebih nilai sehingga dihasilkan nilai yang baru.

Pada Python terdapat beberapa jenis operator, yaitu operator aritmatika, operator string, operator logika, dan operator relasional.

Operator Aritmatika

Operator aritmatika digunakan untuk mengoperasikan nilai dalam bentuk bilangan (bisa bertipe data integer atau float). Dalam sebuah ekspresi matematika, bisa terdapat satu atau lebih operator aritmatika. Misalkan operasi (3+4)*12/(-3-6), terdapat 4 operator aritmatika, yaitu + (penjumlahan), * (perkalian), / (pembagian), dan - (pengurangan).

Secara umum, dalam Python dikenal beberapa operator aritmatika. Perhatikan Tabel 4.2

Tabel 4.2 Operator-operator aritmatika dalam Python

Simbol Operator	Operasi	Contoh	Hasil	Level (presedensi)
**	Perpangkatan	2**3	8	1
%	Modulus (sis hasil bagi)	7%2	1	2
//	Pembagian yang menghasilkan bilangan bulat (pembulatan ke bawah)	10//4	2	
/	Pembagian yang menghasilkan bilangan riil	10/4	2.5	
*	Perkalian	2*7	14	
-	Pengurangan	2-7	-5	3
+	Penjumlahan	2+7	9	

Dari Tabel 4.2 terlihat bahwa operator ** memiliki level presedensi 1, artinya operator ini mendapat prioritas pertama (tingkat presedensi) untuk diselesaikan terlebih dahulu dalam sebuah ekspresi. Selanjutnya operator %, //, /, dan * sama-sama memiliki level ke-2 yang akan diselesaikan berikutnya. Sedangkan operator – dan + memiliki level paling rendah, yang akan dijalankan setelah level ke-2. Sehingga dalam hal ini, apabila diberikan ekspresi sebagai berikut:

$2^{**}3^{*}3-12/2+4$

Akan dihasilkan nilai 22, karena ekspresi tersebut ekuivalen dengan $((2^{**}3)^{*}3)-(12/2)+4$.

Selanjutnya bagaimana dengan ekspresi berikut ini:

$2 + 3 - 7 - 4$

Operator manakah yang dilakukan terlebih dahulu? Operator + dan – memiliki level presedensi yang sama berdasarkan Tabel 4.2. Jika demikian halnya, maka operator yang dilakukan terlebih dahulu adalah mulai dari yang posisinya paling kiri. Sehingga operasi $2 + 3$ dilakukan terlebih dahulu, kemudian hasilnya dikurangi dengan 7, kemudian dikurangi lagi dengan 4. Dengan demikian hasil dari ekspresi tersebut adalah -6.

Operator Untuk String

Dalam subbab sebelumnya disebutkan beberapa operator aritmatika yang hanya berlaku bagi nilai bertipe data angka saja, dalam hal ini khusus untuk tipe data integer dan floating point. Sedangkan pada bagian ini, akan diberikan beberapa operator yang bisa digunakan untuk tipe data string.

Operator String Concatenation (Penggabungan String)

Operator ini digunakan untuk menggabungkan dua string atau lebih dalam Python. Bentuk operatornya adalah menggunakan tanda +. Berikut ini beberapa contohnya:

```
In [2]: 'Hallo' + 'Selamat Pagi?'
```

```
Out[2]: 'HalloSelamat Pagi?'
```

```
In [4]: 'Assalaamu\'alaikum...' + 'Selamat' + 'Pagi?'
```

```
Out[4]: "Assalaamu'alaikum...SelamatPagi?"
```

Contoh ke dua tersebut menggabungkan tiga buah string menjadi satu dengan operator +. Sekaligus menunjukkan cara menuliskan tanda petik ' dalam sebuah string, dalam hal ini string 'Assalaamu'alaikum', yaitu dengan menambahkan tanda \ (backslash) di depan tanda petiknya.

Operator String Replication (Pengulangan String)

Sebuah string bisa ditulis berulang sampai dengan n kali, dengan menggunakan operator *. Berikut ini beberapa contohnya:

```
In [6]: 'Hello' * 3
```

```
Out[6]: 'HelloHelloHello'
```

```
In [8]: 'Hello' * (2+3)
```

```
Out[8]: 'HelloHelloHelloHelloHello'
```

Operator Logika

Operator logika digunakan khusus untuk mengoperasikan nilai bertipe data boolean. Hasil operasinya pun juga dalam bentuk boolean. Operator logika yang bisa digunakan antara lain: `and`, `or`, dan `not`.

Berikut ini beberapa contoh operasinya:

```
In [15]: True and True
```

```
Out[15]: True
```

```
In [13]: False or False
```

```
Out[13]: False
```

```
In [16]: not True or False
```

```
Out[16]: False
```

```
In [18]: not (False and True)
```

```
Out[18]: True
```

Biasanya operasi logika seperti di atas dipakai untuk menyatakan suatu syarat yang digunakan dalam proses percabangan atau perulangan. Pembahasan hal ini akan diberikan di bab yang lain.

Operator Relasional

Operator relasional dalam pemrograman komputer digunakan untuk membandingkan dua buah nilai. Hasil dari operasi yang menggunakan operator relasional adalah berupa nilai boolean. Tabel 4.3 menunjukkan macam-macam operator relasional dan kegunaannya.

Tabel 4.3 Macam-macam operator relasional

Operator	Maksud
>	Membandingkan apakah suatu nilai lebih besar dari nilai yang lain
<	Membandingkan apakah suatu nilai lebih kecil dari nilai yang lain
>=	Membandingkan apakah suatu nilai lebih besar atau sama dengan nilai yang lain
<=	Membandingkan apakah suatu nilai lebih kecil atau sama dengan nilai yang lain
==	Membandingkan apakah suatu nilai sama dengan nilai yang lain
!=	Membandingkan apakah suatu nilai tidak sama dengan nilai yang lain

Operator relasional dapat digunakan di beberapa tipe data, yaitu tipe data angka (integer dan float), tipe data string, dan juga boolean.

Adapun contoh operasi relasional untuk tipe data angka adalah sebagai berikut:

```
In [20]: 3 > 2
```

```
Out[20]: True
```

```
In [21]: 10 <= 9
```

```
Out[21]: False
```

```
In [22]: 8 == (2+6)
```

```
Out[22]: True
```

```
In [23]: 10 != 6
```

```
Out[23]: True
```

Untuk tipe data string, perbandingannya sesuai dengan urutan kode ASCII (*American Standard Code for Information Interchange*). Detil urutan kode ASCII dapat dilihat di <http://www.asciitable.com>

```
In [28]: 'a' < 'c'
```

```
Out[28]: True
```

```
In [29]: 'a' > 'A'
```

```
Out[29]: True
```

```
In [26]: 'APEL' < 'APA'
```

```
Out[26]: False
```

Pada contoh kedua di atas, nilai 'a' lebih besar dari 'A' karena kode ASCII 'a' lebih besar dari 'A'. Sedangkan pada contoh ke tiga dihasilkan nilai False karena seharusnya 'APEL' memiliki nilai yang lebih besar dari 'APA'. Hal ini disebabkan karena pada karakter ke tiga, huruf 'E' dari 'APEL' memiliki nilai yang lebih besar dari karakter ke tiga dari 'APA' yaitu 'A'.

Adapun contoh operasi relasional dari tipe data boolean adalah sebagai berikut.

```
In [30]: True > False
```

```
Out[30]: True
```

```
In [36]: True == (not False)
```

```
Out[36]: True
```

Secara umum, nilai True memiliki nilai yang lebih besar dari False. Dalam hal ini, nilai True biasa direpresentasikan dalam bentuk angka 1, sedangkan False adalah angka 0. Sehingga nilai True lebih besar dari False.

Sama seperti operator logika, biasanya operator relasional ini digunakan untuk menyatakan syarat pada pernyataan bersyarat atau perulangan yang pembahasannya disajikan di bab tersendiri.

Perhatian!! Dalam melakukan operasi, nilai yang dioperasikan haruslah bertipe sama. Sebagai contoh misalkan perintah berikut ini.

```
In [38]: 2 > 'a'
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-38-775563ed3bb4> in <module>()  
----> 1 2 > 'a'  
  
TypeError: '>' not supported between instances of 'int' and 'str'
```

Hasil dari operasi di atas akan muncul error. Hal ini disebabkan karena tipe data dari nilai yang dioperasikan tidak sama, yaitu bertipe angka dan string.

Assignment

Sebuah nilai dapat disimpan ke dalam sebuah variabel. Kegunaan variabel dalam pemrograman komputer dapat diibaratkan seperti kotak penyimpanan. Nilai yang tersimpan dalam sebuah variabel dapat dibaca kembali untuk dioperasikan lagi pada ekspresi lainnya. Istilah khusus proses menyimpan suatu nilai ke dalam variabel adalah *assignment*.

Untuk menyimpan sebuah nilai ke dalam variabel, caranya adalah menggunakan operator *assignment* (=). Contohnya adalah:

```
In [39]: bil1 = 10
```

```
In [40]: bil2 = 20
```

```
In [41]: hasil = bil1 + bil2
```

```
In [42]: print(hasil)
```

30

Pada contoh tersebut, baris pertama menunjukkan perintah untuk menyimpan nilai 10 ke dalam variabel bernama bil1. Sedangkan pada baris ke dua merupakan perintah untuk menyimpan nilai 20 ke dalam variabel bil2. Selanjutnya misalkan ingin menjumlahkan isi dari variabel bil1 dan bil2 maka cukup menjumlahkan kedua variabel tersebut saja. Perintah pada baris ke tiga untuk menyimpan hasil penjumlahan dari kedua nilai variabel ke dalam variabel bernama hasil. Adapun baris perintah yang terakhir untuk menampilkan isi dari variabel hasil nya yang dalam hal ini akan muncul nilai 30 sebagai hasil penjumlahan 10 dan 20.

Sedangkan contoh berikut ini menunjukkan penyimpanan nilai bertipe data string ke dalam variabel kemudian digabung menjadi satu dengan operator string concatenation.

```
In [43]: kata1 = 'Hello'
        kata2 = kata1 + 'World'
        print(kata2)
```

HelloWorld

Sebuah variabel yang sudah terisi dengan suatu nilai dapat diisi lagi dengan nilai lainnya yang baru. Jika demikian hal nya, maka nilai yang baru akan menimpa nilai yang sebelumnya tersimpan dalam variabel tersebut. Hal ini berakibat nilai yang tersimpan sebelumnya tidak bisa dipanggil lagi. Nilai baru yang disimpan dalam sebuah variabel yang sebelumnya telah terisi, tidak harus memiliki tipe data yang sama dengan sebelumnya. Misalnya ada sebuah variabel bernama a yang diisi dengan nilai bertipe integer 1, maka variabel a dapat diisi lagi dengan nilai bertipe data floating point misalnya 1.00001. Perhatikan contoh ini:

```
In [44]: a = 1
```

```
In [45]: print(a)
```

1

```
In [46]: a = 1.00001
```

```
In [47]: print(a)
```

1.00001

Sebuah ekspresi assignment juga bisa melibatkan variabel yang sama dengan ekspresinya. Contohnya adalah sebagai berikut:

```
In [48]: x = 1
x = x + 1
x = x + 1
print(x)
```

3

Maksud dari baris ke dua dari perintah di atas adalah membaca nilai yang tersimpan dari variabel x (dalam hal ini 1) kemudian dijumlahkan dengan 1 sehingga hasilnya 2. Kemudian hasil ini (yaitu 2) disimpan dalam variabel x lagi. Sehingga nilai x adalah 2. Demikian juga untuk perintah pada baris ke tiga.

Di dalam Python operasi aritmatika dan assignment dapat dilakukan sekaligus menggunakan operator assignment lainnya. Sebagai contoh misalkan ada ekspresi $x = x + 3$. Dalam ekspresi tersebut terdapat operator penjumlahan dan operator assignment. Ekspresi tersebut dapat disingkat penulisannya $x += 3$. Secara implisit, makna dari ekspresi $x += 3$ itu sama dengan $x = x + 3$. Operator assignment lainnya yang senada yang bisa digunakan seperti terdapat pada Tabel 4.4

Tabel 4.4 Operator Assignment

Operator Assignment	Contoh dan Maksud
$+=$	$x += 2$ ekuivalen dengan $x = x + 2$
$-=$	$x -= 2$ ekuivalen dengan $x = x - 2$
$*=$	$x *= 2$ ekuivalen dengan $x = x * 2$
$/=$	$x /= 2$ ekuivalen dengan $x = x / 2$
$\% =$	$x \% = 2$ ekuivalen dengan $x = x \% 2$
$**=$	$x ** = 2$ ekuivalen dengan $x = x ** 2$
$// =$	$x // = 2$ ekuivalen dengan $x = x // 2$

Aturan Pemberian Nama Variabel

Pemberian nama sebuah variabel pada prinsipnya bisa sembarang, asalkan memenuhi ketentuan sebagai berikut:

1. Tidak boleh diawali dengan angka
2. Tidak boleh dipisahkan dengan spasi
3. Hanya boleh terdiri dari angka, huruf, dan underscore

Meskipun pemberian nama variabel adalah sembarang, namun sangat disarankan sesuai dengan maksud nilai yang akan disimpan di dalamnya. Perhatikan contoh rangkaian ekspresi berikut ini untuk menghitung luas segitiga dengan panjang alas 3 dan tinggi 7 satuan luas melalui Python Shell.

```
In [50]: # script hitung Luas segitiga
# dg alas 3 dan tinggi 7 satuan

alasSegitiga = 3
tinggiSegitiga = 7
luasSegitiga = alasSegitiga * tinggiSegitiga * 0.5
print(luasSegitiga)

10.5
```

Sifat Case Sensitivitas Variabel

Di dalam Python, besar kecilnya huruf dalam penulisan nama variabel akan berpengaruh (case sensitive). Sebagai contoh misalkan variabel `alasSegitiga` dengan `AlasSegitiga` adalah dua variabel yang berbeda. Perhatikan contoh berikut ini

```
In [51]: alasSegitiga = 10
tinggiSegitiga = 5
luasSegitiga = AlasSegitiga * tinggiSegitiga * 0.5
print(luasSegitiga)

-----
NameError                                Traceback (most recent call last)
<ipython-input-51-80d5c075436e> in <module>()
      1 alasSegitiga = 10
      2 tinggiSegitiga = 5
----> 3 luasSegitiga = AlasSegitiga * tinggiSegitiga * 0.5
      4 print(luasSegitiga)

NameError: name 'AlasSegitiga' is not defined
```

Dari contoh di atas tampak bahwa akan muncul error di dalam Python yang menunjukkan variabel `AlasSegitiga` belum didefinisikan sebelumnya (tidak dikenal). Hal ini disebabkan pada baris pertama penulisan pada variabel `alasSegitiga` menggunakan huruf a kecil di bagian paling depan. Sedangkan pada baris ke tiga ditulis dengan dengan huruf a besar.